

Class 07

Subqueries & Set Operations

◆ 1. Subqueries (Nested Queries)

A **subquery** is a query **inside another query**.

It helps to get intermediate results used by the main query.

Example:

```
SELECT name, salary  
FROM employees  
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Here, the inner query finds the average salary,
and the outer query lists all employees earning above that average.

Types of Subqueries:

◆ (a) Subquery in WHERE Clause

Used to compare values using **IN**, **ANY**, **ALL**, or comparison operators.

```
SELECT name  
FROM employees  
WHERE department_id IN (SELECT id FROM departments  
WHERE location = 'Delhi');
```

- **IN** → Matches any value in the subquery result.
- **ANY** → True if condition is true for *any* returned value.
- **ALL** → True only if condition is true for *all* returned values.

◆ (b) Subquery in FROM Clause (Derived Table)

A subquery can act as a **temporary table**.

```
SELECT dept, AVG(avg_salary)
FROM (
  SELECT department AS dept, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department
) AS temp
GROUP BY dept;
```

The subquery `temp` acts like a virtual table.

◆ (c) Subquery in SELECT Clause (Scalar Subquery)

Returns a **single value** (one row, one column).

```
SELECT name,  
       (SELECT COUNT(*) FROM projects WHERE projects.emp_id = employees.id) AS total_projects  
FROM employees;
```

Returns each employee's name along with number of projects they're handling.

◆ 2. Set Operators

Set operators are used to **combine results** of two or more SELECT statements.

◆ UNION

Combines results from two queries and **removes duplicates**.

```
SELECT city FROM customers  
UNION  
SELECT city FROM suppliers;
```

Returns all unique cities from both tables.

◆ UNION ALL

Combines results from two queries **without removing duplicates**.

```
SELECT city FROM customers  
UNION ALL  
SELECT city FROM suppliers;
```

Operator	Removes Duplicates	Performance
UNION	Yes	Slower
UNION ALL	No	Faster

◆ 3. Window Functions (Analytic Functions) – Basic Intro

Window functions perform calculations **across a set of table rows** related to the current row — without collapsing the result into one row.

Example:

```
SELECT name, department, salary,  
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank  
FROM employees;
```

- ROW_NUMBER() → Sequential number per row
- RANK() → Ranking with gaps (1, 2, 2, 4...)
- DENSE_RANK() → Ranking without gaps (1, 2, 2, 3...)

◆ 4. Common Table Expressions (CTEs)

A **CTE** provides a way to create a temporary result set that can be referred to within a SELECT, INSERT, UPDATE, or DELETE statement.

Example:

```
WITH dept_avg AS (  
  SELECT department_id, AVG(salary) AS avg_salary  
  FROM employees  
  GROUP BY department_id  
)  
SELECT e.name, e.salary, d.avg_salary  
FROM employees e  
JOIN dept_avg d ON e.department_id = d.department_id;
```

CTE dept_avg behaves like a virtual named query used later in the main query.

◆ 5. Views

A **View** is a **virtual table** based on a postgresSQL query result. It doesn't store data — it just saves the query logic.

Example:

```
CREATE VIEW employee_summary AS  
SELECT name, department, salary  
FROM employees  
WHERE salary > 50000;
```

To use the view:

```
SELECT * FROM employee_summary;
```